Assignment - 05

Unit 5: NoSQL Database (Riak and Neo4j) | BCA305T

Q1. Explain the architecture and working of Riak as a Key-Value NoSQL database. Discuss how Riak stores data using keys and values and explain the major characteristics that distinguish it from relational databases.

Answer:

Riak (Riak KV) is a distributed, open-source key-value NoSQL database inspired by Amazon's *Dynamo* paper and written mainly in Erlang. It is built to be highly available, fault tolerant and easy to scale, and it has a **masterless** design: every node in the cluster is equal and can serve any request.

Architecture

- **Cluster and nodes:** a cluster is a set of physical nodes. Any node can act as the *coordinating node* for a client request.
- **Ring and partitions:** the key space is a fixed ring (default 64 partitions). Each key is hashed (SHA-1) and mapped to a partition. Nodes own several partitions through **virtual nodes (vnodes)**.
- **Replication:** each object is copied to *N* vnodes (default n_val = 3), placed on different physical nodes where possible.
- **Gossip protocol:** nodes exchange cluster state periodically, so no central metadata server is needed.
- **Pluggable storage backends:** Bitcask (fast, in-memory key directory), LevelDB / Leveled (supports secondary indexes, large data sets) and Memory.
- **Failure handling:** hinted handoff, read repair and active anti-entropy (Merkle trees) repair data after failures.

How data is stored

Data is organised as **bucket type / bucket / key / value**. A bucket is only a namespace, not a table. The key is a unique string, and the value is an opaque object (JSON, text, image, binary) with metadata such as content-type, vector clock and optional secondary index entries.

```
PUT  /types/default/buckets/users/keys/user101
Content-Type: application/json
{ "name": "Rahul", "city": "Roorkee" }

GET    /types/default/buckets/users/keys/user101
DELETE /types/default/buckets/users/keys/user101
```

Working of a write and read

- The client sends the request to any node; that node hashes the key and finds the N partitions that own it.
- **Write:** the object is sent to all N replicas; the client gets success once *w* replicas acknowledge.
- **Read:** the coordinator asks all N replicas and returns the result when *r* replicas answer; if replicas disagree, the newest version is returned and stale copies are repaired (read repair).

Characteristics that distinguish Riak from relational databases

Aspect	Relational DB	Riak
Data model	Tables, rows, columns	Key to value (opaque object)

Aspect	Relational DB	Riak
Schema	Fixed schema	Schema-less; value structure decided by application
Query	SQL, joins, ad hoc queries	Primarily get/put/delete by key
Architecture	Usually single master	Masterless, peer-to-peer ring
Scaling	Vertical mainly	Horizontal: just add nodes
Consistency	Strong (ACID)	Tunable, eventual by default (BASE)
Availability	Failover needed	Always writable even during failures
Relationships	Foreign keys and joins	Handled by the application

Q2. Explain the Key-Value Data Model in detail. Discuss the structure of data, key-value store features, data access mechanism, and limitations of this model. Why is a Key-Value database suitable for applications requiring fast lookups?

Answer:

The **key-value data model** is the simplest NoSQL model. It works like a giant hash table or dictionary: every item is stored as a pair of a unique **key** and a **value**. The database does not interpret the value; only the key is used to find it.

Structure of data

- **Key:** a unique identifier such as *user:101* or *session:af39c*. Good keys are short, meaningful and often use prefixes.
- **Value:** any data - number, string, JSON, XML, list, image or binary blob; the store treats it as opaque.
- **Namespace:** keys are grouped in buckets (or databases) to avoid clashes.

```
Key             Value
user:101        {"name":"Rahul","age":21}
session:af39c   {"uid":101,"expires":"2026-11-01T10:00"}
cart:101        [{"item":"P100","qty":2}]
```

Features

- Very fast reads and writes (usually O(1) lookup by hash).
- Simple, schema-free model and simple API.
- Easy horizontal scaling through partitioning of keys.
- Built-in replication and high availability in distributed stores.
- Optional features: TTL expiry, in-memory caching, atomic counters, sets.

Data access mechanism

All access is through a small set of operations: **PUT/SET(key, value)**, **GET(key)** and **DELETE(key)**. The key is hashed to find the node and location holding the value, so no table scan or join is needed.

Limitations

- No query on value contents; you must know the key (unless secondary indexes or search are added).
- No joins and weak support for relationships between items.
- Limited or no multi-key transactions.
- No schema or constraints, so data quality depends on the application.

- Complex analytics and aggregation are difficult.
- Updating part of a large value usually means rewriting the whole value.

Why suitable for fast lookups

Because access is by hash of the key, the lookup cost is almost constant regardless of data size. There is no query parsing, join or optimizer overhead, data can be kept in memory or on fast storage, and the load spreads evenly over nodes. This makes the model ideal for sessions, caches, user profiles, shopping carts and configuration data.

Q3. Analyze the consistency model of Riak. Explain how Riak handles consistency in a distributed environment and discuss the trade-off between consistency, availability, and partition tolerance with a suitable example.

Answer:

Riak is designed as an **AP system** in the CAP theorem: by default it favours **availability** and **partition tolerance** and gives **eventual consistency**. Replicas may briefly hold different versions, but they converge over time. Consistency is **tunable** per request.

How Riak handles consistency

- **Quorum parameters:** N = number of replicas, R = replicas needed for a read, W = replicas needed for a write. If R + W is greater than N, reads normally see the latest write (quorum). Shortcuts include *one*, *quorum* and *all*.
- **Sloppy quorum and hinted handoff:** if a replica node is down, another node temporarily accepts the write and later hands it back, so writes still succeed.
- **Vector clocks / dotted version vectors:** track the causal history of each object so Riak can tell whether one version replaces another or whether they are concurrent (conflict).
- **Siblings:** with *allow_mult = true*, concurrent updates are kept as sibling values and the application (or a CRDT) resolves them on the next read.
- **Last-write-wins:** optional setting where the value with the newest timestamp wins (simple, but can lose updates).
- **CRDTs (Riak data types):** counters, sets, maps, flags and registers merge automatically without conflicts.
- **Read repair and active anti-entropy:** stale replicas are fixed during reads and by background Merkle-tree comparison.
- **Strong consistency (optional):** from Riak 2.0 a bucket type can be made strongly consistent using a consensus protocol, at the cost of availability during partitions.

CAP trade-off

CAP says that during a network **partition** a system must choose between **consistency** and **availability**. Riak (default) keeps accepting reads and writes on both sides of the partition (availability) and reconciles later (eventual consistency). A CP system would reject some requests to keep data identical everywhere.

Example

In a shopping site, user Rahul adds item A to his cart from his phone while the network between two data centres is cut, and adds item B from his laptop on the other side. With Riak, both writes succeed (available). The two cart versions become siblings; when the network heals, the application (or a Riak Set data type) merges them into a cart containing A and B. Rahul never saw an error. In a strictly consistent system one of the writes might have been refused during the partition.

Setting	Effect
W = 1, R = 1	Fastest, highest availability, weakest consistency
W = quorum, R = quorum	Balanced; usually reads latest write
W = all, R = all	Strongest consistency, lowest availability

Q4. Discuss transactions and multi-operation transactions in Riak. Why are traditional relational-style transactions difficult to implement in distributed Key-Value databases? Explain the possible approaches for maintaining data correctness across multiple operations.

Answer:

A **transaction** groups operations so they either all succeed or all fail (ACID). In Riak, the only atomic unit is a **single key operation** (a put or get of one object). Riak does **not** support multi-key or multi-operation ACID transactions with rollback. Writes are replicated with a write quorum, but there is no 'begin / commit / rollback'.

Why relational-style transactions are difficult in distributed key-value stores

- **Data spread over many nodes:** keys used in one transaction may live on different nodes and partitions, so coordination across them is needed.
- **Coordination cost:** protocols like two-phase commit need several network round trips and locks, reducing speed and scalability.
- **CAP limits:** during a partition, guaranteeing atomic and consistent multi-key updates conflicts with staying available.
- **Node failures:** a coordinator crash in the middle of a commit can leave participants blocked or inconsistent.
- **Masterless design:** there is no single node that can order or lock all operations.
- **Opaque values:** the store does not understand the data, so it cannot enforce constraints or isolation rules.

Approaches to keep data correct across multiple operations

- **Design for a single key:** put related data into one value (e.g., the whole cart or order as one JSON object) so one atomic write is enough.
- **Quorum tuning:** use higher W and R (or strong-consistency bucket types) for critical data.
- **Conflict resolution:** use vector clocks with siblings and merge logic in the application.
- **CRDTs:** counters, sets and maps that merge safely without coordination.
- **Compensating transactions / Saga pattern:** run steps one by one and, if a later step fails, run an undo step for the earlier ones.
- **Idempotent operations and retries:** design operations so repeating them gives the same result.
- **Application-level two-phase commit or write-ahead record:** store a 'pending operation' key, apply steps, then mark it complete; a recovery job finishes unfinished work.
- **Eventual consistency by design:** accept short inconsistency and rely on background repair.

```
// Saga style: transfer 500 from A to B
1. put  tx:991  = {from:A, to:B, amt:500, state:"PENDING"}
2. debit  account:A  (idempotent, tagged with tx id 991)
3. credit account:B  (idempotent, tagged with tx id 991)
4. update tx:991 state = "DONE"
```

```
if step 3 fails -> compensate: credit account:A back, state = "CANCELLED"
```

Q5. Explain the query features of Riak. How does querying by data differ from querying by key? Discuss how Riak can support searching, secondary indexes, and data retrieval when the application requirements become more complex.

Answer:

Query by key

The primary and fastest method in Riak is to fetch an object by its **bucket and key** (GET). The key is hashed and the request goes straight to the right replicas. This is O(1), needs no index, and is the recommended way to access data.

Query by data (by value contents)

Querying by data means finding objects based on what is *inside* the value (for example, all users living in Roorkee). A plain key-value store cannot do this because values are opaque; without extra features it would need a full scan of every key, which is slow and discouraged in production. Riak therefore adds extra mechanisms.

Aspect	Query by key	Query by data
Input	Exact key	Conditions on value or metadata
Speed	Constant, very fast	Slower; needs index or search
Mechanism	Hash lookup	Secondary index, Riak Search, MapReduce
Results	One object	Zero, one or many objects

Riak mechanisms for richer queries

- **Secondary indexes (2i):** tag an object with index fields such as *city_bin* (string) or *age_int* (integer). Riak can then look up exact matches or ranges. Needs a backend that supports it (LevelDB / Leveled).
- **Riak Search (Solr-based):** full-text indexing of JSON, XML or text values with schemas; supports wildcards, boolean and range queries, and faceting.
- **MapReduce:** run map and reduce functions (JavaScript or Erlang) over a set of keys or an index result for aggregation and analysis; heavy, so used sparingly.
- **Key listing:** listing all keys in a bucket is possible but very expensive, so should be avoided in production.
- **Data types:** sets and maps can store lists of related keys that act as manual indexes.
- **Application-level design:** build meaningful keys (e.g., *order:2026:10:30:991*) or maintain your own index objects.

```
// add a secondary index when storing
PUT /buckets/users/keys/u101
x-riak-index-city_bin: Roorkee
x-riak-index-age_int: 21

// query by index
GET /buckets/users/index/city_bin/Roorkee
GET /buckets/users/index/age_int/18/25      // range
```

When requirements grow more complex (joins, deep analytics, graph-style queries), Riak is usually combined with other tools such as Solr search, Spark analytics, or a relational or graph database.

Q6. A web application needs to store millions of user sessions where each session contains user ID, login time, device information, and session expiry

information. Explain how Riak can be used to store session information and discuss the advantages of using a Key-Value database for this application.

Answer:

A session is always accessed by its **session ID** (from the cookie), which makes it an ideal key-value use case. Riak stores each session as one object, with the session ID as the key.

Data design

```
Bucket: sessions
Key   : sess:9f3a7c21b0d4        (random, unique session ID)
Value :
{
  "userId": 101,
  "loginTime": "2026-10-30T09:15:00Z",
  "device": { "type": "mobile", "os": "Android 14", "browser": "Chrome" },
  "ip": "103.21.4.9",
  "expiresAt": "2026-10-30T10:15:00Z"
}
```

Operations

- **Login:** generate session ID, PUT the object.
- **Each request:** GET the key to validate the user; optionally PUT to extend *expiresAt* (sliding expiration).
- **Logout:** DELETE the key.
- **Expiry:** check *expiresAt* on read, or use a backend with TTL support (Bitcask *expiry_secs*) so old sessions are removed automatically.
- **Find all sessions of a user (optional):** add a secondary index *userId_int*, for example to log a user out from all devices.

Advantages of a key-value database here

- **Very low latency:** a single hashed lookup per request, ideal for checks on every page.
- **Millions of sessions:** data spreads over nodes, and more nodes can be added as users grow.
- **High availability:** replication (N = 3) and masterless design mean users stay logged in even if a node fails.
- **Simple model:** no joins or schema changes needed; the session object can gain new fields freely.
- **Automatic expiry:** TTL support suits temporary data.
- **High write throughput:** sessions are created and updated constantly; Riak's append-only Bitcask backend handles this well.
- **Eventual consistency is acceptable:** a session being slightly stale for milliseconds is not critical.

Q7. An e-commerce application needs to maintain user profiles, preferences, and shopping cart data. Design a conceptual Key-Value data structure for these requirements and explain how the data can be efficiently accessed and updated.

Answer:

In a key-value design, each kind of data gets its own bucket, and the key is built from the owner's ID so that data can be found directly without searching.

Conceptual design

```
Bucket: profiles        Key: user:101
{ "name": "Sneha", "email": "sneha@mail.com", "phone": "98xxxxxx01",
  "addresses": [ {"type":"home","city":"Aligarh","pin":202001} ],
```

```
  "createdAt": "2026-01-10" }

Bucket: preferences     Key: user:101
{ "language": "en", "currency": "INR", "theme": "dark",
  "categories": ["Mobiles","Books"], "notifications": {"email":true,"sms":false} }

Bucket: carts           Key: user:101
{ "items": [ {"productId":"P100","qty":1,"price":19999},
             {"productId":"P200","qty":2,"price":799} ],
  "updatedAt": "2026-10-30T09:20:00Z" }
```

Why this design is efficient

- **Same key in different buckets:** user:101 is used in profiles, preferences and carts, so any one can be read with one GET and no join.
- **One value per cart:** the entire cart is read or written in one atomic operation.
- **Separate buckets:** data with different update frequency and size is separated (cart changes often, profile rarely). Preferences could also be merged into the profile object if always read together.
- **Small values:** keeps network and storage cost low.

Access and update

- **Read profile:** GET profiles/user:101.
- **Update preference:** GET, change the field, PUT back the object (or use a Riak Map data type to update one field without rewriting everything).
- **Add to cart:** GET cart, add item, PUT cart. Using a Riak *Map* or *Set* CRDT lets two devices add items at the same time and merge safely.
- **Checkout:** read cart, create an order object (key *order:991*), then DELETE or clear the cart.
- **Guest carts:** use a cart ID key (cart:guest-77a) and merge into the user's cart at login.
- **Expiry:** abandoned carts can expire using TTL.

Q8. Explain relationships among data in Key-Value databases. Why are relationships more difficult to represent in a simple Key-Value model than in a relational or graph database? Discuss suitable techniques for handling related data.

Answer:

Relationships describe how data items are connected, for example a customer places orders, an order contains products, or a user follows other users. In key-value databases the store only knows keys and opaque values, so it has no built-in notion of relationships.

Why relationships are difficult

- **No foreign keys or constraints:** the database cannot check that a referenced key exists or enforce referential integrity.
- **No joins:** related data must be fetched with several separate GETs and combined in the application.
- **Opaque values:** the database cannot look inside a value to follow a reference.
- **Reverse lookups are hard:** finding 'all orders of customer 101' needs an index or a maintained list, because you cannot query by value.
- **Many-to-many links** must be maintained manually in both directions, risking inconsistency without transactions.

- **Multi-hop traversal** (friends of friends) means many round trips, unlike graph databases where relationships are stored as direct pointers, or relational databases with join operators and foreign key indexes.

Techniques for handling related data

- **Embed related data** in one value (e.g., order items inside the order object) for one-to-few relations.
- **Store keys as references:** keep the related key inside the value, such as *customerId: 101* in the order, and fetch it with a second GET.
- **Composite or structured keys:** encode the relationship in the key, e.g., *customer:101:orders* or *order:101:991*.
- **Link lists / index objects:** store a list of keys, e.g., *customer:101:orders = [991, 992, 993]*, kept updated by the application (a Riak Set data type works well).
- **Secondary indexes:** index the foreign key field (*customerId_int*) for reverse lookups.
- **Riak Links (older feature) and Riak Search:** link objects or search on reference fields.
- **Denormalization:** duplicate small frequently-read data (e.g., customer name in the order) to avoid extra reads, accepting some redundancy.
- **Use a better-suited database** for relationship-heavy parts (graph database for social links, relational for joins).

```
customer:101              -> {"name":"Sneha"}
customer:101:orders       -> ["order:991","order:992"]
order:991                 -> {"customerId":"customer:101","items":[...],"total":19999}
```

Q9. Explain scaling in Riak. Discuss how distributed Key-Value databases handle increasing data volume and user requests. Explain the role of partitioning, replication, and distributed nodes in achieving scalability and fault tolerance.

Answer:

Scaling means handling more data and more requests. Riak is designed for **horizontal scaling** (scale out): capacity grows by adding commodity nodes rather than buying one bigger machine.

Role of partitioning

- Keys are hashed with **consistent hashing** onto a ring divided into fixed partitions (default 64, often more).
- Each node owns many partitions (vnodes), so data and load are spread evenly.
- When a node is **added**, it takes over some partitions from existing nodes; only that portion of data moves. When a node is removed, its partitions are redistributed.
- Because there is no central master, there is no bottleneck.

Role of replication

- Each object is stored on N replicas (default 3) on different nodes.
- Reads can be served by any replica, spreading the read load.
- If a node fails, other replicas still hold the data, so no data loss and no downtime.
- Multi-datacenter replication (in the commercial edition) copies data across sites for disaster recovery and local access.

Role of distributed nodes

- Every node can accept requests, so throughput grows almost **linearly** with the number of nodes.

- **Gossip protocol** shares ring state so all nodes know where data lives.
- **Hinted handoff** keeps writes working while a node is down and returns data when it recovers.
- **Read repair and active anti-entropy** repair differences between replicas.
- **Rebalancing** is automatic when nodes join or leave.

Scalability and fault tolerance together

Mechanism	Scalability	Fault tolerance
Partitioning (vnodes)	Spreads data and load	Failure affects only some partitions
Replication (N copies)	Distributes reads	Data survives node failure
Masterless nodes	No single bottleneck	No single point of failure
Hinted handoff / repair	Keeps cluster smooth when growing	Restores consistency after failure

The same ideas (consistent hashing, replication and peer nodes) are used by other distributed key-value stores such as Amazon DynamoDB, Cassandra and Voldemort.

Q10. Explain the concept of Operations by Sets in Key-Value databases. Discuss why set-based operations can be useful for applications such as user groups, tags, product categories, and social relationships. Give suitable examples.

Answer:

Operations by sets means performing mathematical set operations on collections of keys or members - **union**, **intersection**, **difference**, membership test, add and remove - instead of retrieving and comparing items one at a time. A set holds unique, unordered members. Some key-value stores (such as Redis with SADD, SINTER, SUNION, SDIFF) support them natively; in Riak, the **Set data type (CRDT)** supports add, remove and membership, and Riak Search or secondary indexes can return key sets that the application intersects or unions. Riak's older *link walking* and MapReduce can also work on sets of keys.

Basic set operations

Operation	Meaning	Example
Add / Remove	Insert or delete a member	Add 'nosql' to tags
Membership	Is x in the set?	Is user 7 in group Admins?
Union (A or B)	Members in either set	Users in group A or B
Intersection (A and B)	Members in both	Common friends of two users
Difference (A minus B)	In A but not B	Products in Sale but not in Electronics

Usefulness

- **Uniqueness:** duplicates are avoided automatically (a tag or user can appear only once).
- **Speed:** set operations are done close to the data, avoiding many round trips.
- **Conflict-free merging:** Riak Set CRDTs merge concurrent adds and removes from different nodes safely.
- **Natural modelling** of groups, memberships and categories.

Examples

```
// User groups
group:admins     = {u1, u5, u9}
```

```
group:editors    = {u5, u7}
Intersection(admins, editors) = {u5}       // users in both groups

// Tags
tag:mongodb = {post1, post4, post8}
tag:nosql   = {post4, post8, post9}
Intersection = {post4, post8}              // posts tagged both

// Product categories
category:mobiles = {P1, P2, P3};  category:sale = {P2, P3, P7}
Intersection = {P2, P3}                    // mobile phones on sale

// Social relationships
friends:rahul = {amit, neha, sam};  friends:sneha = {neha, sam, ravi}
Intersection = {neha, sam}                 // mutual friends
Difference(friends:sneha, friends:rahul) = {ravi}   // friend suggestion
```

Q11. Explain Graph Databases and their data model. Discuss the concepts of nodes, relationships/edges, properties, and labels and explain why graph databases such as Neo4j are particularly suitable for highly connected data.

Answer:

A **graph database** stores data as a **graph** - entities and the connections between them - and treats relationships as first-class data. **Neo4j** is the most popular graph database and uses the **property graph model**.

Data model concepts

- **Nodes:** entities such as a person, product or city (like rows or objects).
- **Relationships (edges):** directed, named connections between two nodes, e.g., *FRIEND_OF*, *BOUGHT*, *LIVES_IN*. Every relationship has a type and a direction and is stored physically with the nodes.
- **Properties:** key-value pairs attached to both nodes and relationships, e.g., a Person node has name and age, and a FRIEND_OF relationship has *since: 2020*.
- **Labels:** tags that group nodes into types (e.g., :Person, :Product), used for querying and indexing. A node can have several labels.

```
(:Person {name:"Rahul", age:21})-[:FRIEND_OF {since:2020}]->(:Person {name:"Sneha"})
(:Person {name:"Sneha"})-[:BOUGHT {date:"2026-10-01"}]->(:Product {name:"Phone X"})
```

Why Neo4j suits highly connected data

- **Index-free adjacency:** each node stores direct pointers to its neighbours, so moving along a relationship takes constant time, regardless of total data size.
- **Fast traversals:** multi-hop queries (friends of friends, routes, recommendation paths) are fast, whereas relational joins slow down as depth and data grow.
- **Natural modelling:** the data model looks like the whiteboard diagram, so it is easy to understand and change.
- **Flexible schema:** new node types, relationships and properties can be added without migration.
- **Expressive query language (Cypher)** for patterns.
- **Typical uses:** social networks, recommendation engines, fraud detection, network and IT operations, knowledge graphs, route finding.

Q12. Explain the major features of Neo4j as a Graph NoSQL database. Discuss its consistency, transactions, availability, query capabilities, and scaling

mechanisms. Compare its approach with that of a traditional relational database.

Answer:

Major features

- Native property graph storage and processing with index-free adjacency.
- **Cypher** declarative query language; supports Bolt protocol and drivers for many languages.
- Flexible, schema-optional model with constraints and indexes available.
- Graph Data Science library, visualization tools (Neo4j Browser / Bloom), import tools and APOC procedures.

Consistency and transactions

- Neo4j is fully **ACID** compliant: every write runs in a transaction with atomicity, consistency, isolation (read-committed by default) and durability using a write-ahead transaction log.
- In a cluster, writes go through the leader and are committed once a majority of primaries confirm (Raft consensus), giving strong consistency for writes. Read replicas (secondaries) are eventually consistent, but *bookmarks* give causal consistency, so a client can read its own writes.

Availability

- **Clustering** (using Raft) with several primary servers: if the leader fails, another is elected automatically.
- Secondary servers (read replicas) copy data from primaries and provide extra read capacity and resilience.
- Online backups and multi-region deployments are possible.

Query capabilities

- Pattern matching with Cypher (MATCH, WHERE, RETURN), variable-length path queries, shortest path, aggregation, ordering and full-text indexes.
- Graph algorithms (PageRank, community detection, similarity) through the Graph Data Science library.

Scaling

- **Vertical scaling** of a single server is very effective because the engine is efficient and uses a page cache.
- **Read scaling** by adding read replicas (secondary servers).
- **Horizontal data scaling** through **Fabric** / composite databases and sharding of data by property or domain, though graphs are harder to partition than key-value data because relationships cross partitions.

Comparison with a relational database

Aspect	Relational DB	Neo4j
Model	Tables, rows, foreign keys	Nodes, relationships, properties
Relationships	Computed at query time with joins	Stored directly; traversed through pointers
Schema	Fixed schema	Flexible, schema-optional
Query language	SQL	Cypher
Deep or multi-hop queries	Slower as joins multiply	Fast; cost depends on traversed part only

Aspect	Relational DB	Neo4j
Transactions	ACID	ACID
Scaling	Vertical mainly, sharding is complex	Vertical, read replicas, Fabric sharding
Best for	Tabular, transactional, reporting data	Highly connected data

Q13. Explain graph database query processing using Neo4j. Discuss how graph traversal and pattern-based queries can be used to find relationships among data. Why can graph-based querying be more effective than relational joins for highly connected datasets?

Answer:

Neo4j queries are written in **Cypher**, a declarative language that describes a **pattern** using ASCII art: nodes in round brackets and relationships in square brackets with arrows. The query engine finds a starting node (using a label index) and then **traverses** relationships to locate every part of the graph that matches the pattern.

Query processing steps

- **Parse and plan:** the Cypher query is parsed and the planner chooses an efficient execution plan.
- **Anchor:** find start nodes using a label scan or index (e.g., Person with name 'Rahul').
- **Expand:** follow relationships from node to node using direct pointers (index-free adjacency).
- **Filter and return:** apply WHERE conditions and return nodes, relationships or paths.

Pattern and traversal examples

```
// 1. Direct friends of Rahul
MATCH (r:Person {name:"Rahul"})-[:FRIEND_OF]->(f:Person)
RETURN f.name

// 2. Friends of friends (2 hops) not already friends
MATCH (r:Person {name:"Rahul"})-[:FRIEND_OF*2]->(fof:Person)
WHERE NOT (r)-[:FRIEND_OF]->(fof) AND fof <> r
RETURN DISTINCT fof.name

// 3. Shortest path between two people
MATCH p = shortestPath((a:Person {name:"Rahul"})-[:FRIEND_OF*..6]-(b:Person {name:"Ravi"}))
RETURN p

// 4. Recommend products bought by friends
MATCH (r:Person {name:"Rahul"})-[:FRIEND_OF]->(:Person)-[:BOUGHT]->(p:Product)
WHERE NOT (r)-[:BOUGHT]->(p)
RETURN p.name, count(*) AS score ORDER BY score DESC
```

Why graph querying beats relational joins for connected data

- **Joins are computed at query time:** a relational database must match foreign keys through indexes for every join, and the cost grows with table size and number of joins (each extra hop adds another join).
- **Graph traversal is local:** Neo4j only touches the part of the graph reachable from the starting node, so time depends on the amount of data traversed, not on total database size.
- **Constant-time hops:** index-free adjacency gives direct pointers to neighbours.
- **Variable-depth queries:** paths of unknown length are natural in Cypher but need recursive SQL in a relational database.
- **Simpler queries:** a four-hop recommendation needs a short Cypher pattern but many joins in SQL.

- **Many-to-many relationships** need no join tables.

For example, finding friends-of-friends-of-friends in a social network of a million users is fast in Neo4j, while the same query with a self-join on a friendship table slows down sharply with each additional level.

Q14. A social networking application needs to represent users, friendships, followers, posts, likes, groups, and recommendations. Explain how you would model this application using Neo4j. Discuss the advantages of using a graph database for relationship-based queries.

Answer:

Graph model

Node label	Key properties
:User	userId, name, email, city, joinedOn
:Post	postId, text, createdAt
:Group	groupId, name, description
:Tag	name

Relationship	Meaning / properties
(:User)-[:FRIEND_OF {since}]-(:User)	Friendship (mutual)
(:User)-[:FOLLOWS]->(:User)	Directed follow
(:User)-[:POSTED {at}]->(:Post)	Author of a post
(:User)-[:LIKES {at}]->(:Post)	Like
(:User)-[:COMMENTED {text}]->(:Post)	Comment
(:User)-[:MEMBER_OF {role}]->(:Group)	Group membership
(:Post)-[:TAGGED]->(:Tag)	Post topic
(:User)-[:INTERESTED_IN]->(:Tag)	Interest

```
CREATE CONSTRAINT FOR (u:User) REQUIRE u.userId IS UNIQUE;

CREATE (r:User {userId:1, name:"Rahul"}), (s:User {userId:2, name:"Sneha"}),
       (p:Post {postId:10, text:"Learning Neo4j"}), (g:Group {name:"BCA 2026"}),
       (r)-[:FRIEND_OF {since:2020}]->(s),
       (s)-[:FOLLOWS]->(r),
       (r)-[:POSTED]->(p),
       (s)-[:LIKES]->(p),
       (s)-[:MEMBER_OF]->(g), (r)-[:MEMBER_OF]->(g);
```

Example queries

```
// News feed: recent posts of people I follow
MATCH (me:User {userId:1})-[:FOLLOWS]->(f)-[:POSTED]->(p:Post)
RETURN f.name, p.text ORDER BY p.createdAt DESC LIMIT 20

// Friend recommendation: people with most mutual friends
MATCH (me:User {userId:1})-[:FRIEND_OF]-(m)-[:FRIEND_OF]-(rec)
WHERE rec <> me AND NOT (me)-[:FRIEND_OF]-(rec)
RETURN rec.name, count(m) AS mutualFriends ORDER BY mutualFriends DESC LIMIT 5

// Group recommendation: groups joined by my friends
MATCH (me:User {userId:1})-[:FRIEND_OF]-(f)-[:MEMBER_OF]->(g:Group)
WHERE NOT (me)-[:MEMBER_OF]->(g)
RETURN g.name, count(f) AS friendsInGroup ORDER BY friendsInGroup DESC
```

```
// Who liked posts of a given user
MATCH (:User {userId:1})-[:POSTED]->(p)<-[:LIKES]-(u) RETURN u.name, count(p)
```

Advantages of a graph database here

- **Fast multi-hop queries** such as mutual friends, friends of friends and feeds, without costly joins.
- **Recommendations:** collaborative filtering patterns (people who like what my friends like) are simple Cypher queries.
- **Natural, intuitive model** that matches how the application thinks about the network.
- **Flexible evolution:** new relationship types (e.g., BLOCKED, SHARED) can be added without changing schema.
- **Real-time performance** as the network grows because traversals touch only local neighbourhoods.
- **Analytics:** influence ranking, community detection and fake-account detection through graph algorithms.
- **ACID transactions** keep the connections consistent (e.g., follow and counter update together).

Q15. Compare Riak Key-Value databases and Neo4j Graph databases with respect to data structure, consistency, transactions, availability, query features, relationships, scalability, and suitable use cases. Based on these characteristics, recommend which database would be more appropriate for a shopping-cart system, user-profile system, and social-network recommendation system, with proper justification.

Answer:

Comparison

Aspect	Riak (Key-Value)	Neo4j (Graph)
Data structure	Key to opaque value in buckets	Nodes, relationships and properties with labels
Consistency	Eventual by default; tunable quorum; optional strong consistency	Strong (ACID) for writes; read replicas eventually consistent with causal bookmarks
Transactions	Only single-key atomic operations; no multi-key ACID	Full ACID multi-operation transactions
Availability	Very high; masterless, sloppy quorum, hinted handoff, always writable	High through clustering with leader election; may pause writes if majority lost
Query features	Get/put/delete by key; secondary indexes, Riak Search, MapReduce	Cypher pattern matching, traversals, shortest path, graph algorithms
Relationships	Not native; managed by application using key references	First-class, stored and traversed directly
Scalability	Excellent horizontal scaling by consistent hashing and adding nodes	Strong vertical and read scaling; sharding is harder because of connections
Speed	O(1) lookups by key	Fast relationship traversals
Use cases	Sessions, carts, profiles, caching, logging, IoT, high-volume simple data	Social networks, recommendations, fraud detection, knowledge graphs, routing

Recommendations

System	Best choice	Justification
Shopping cart	**Riak**	Cart is always accessed by user or cart ID, needs very fast reads and writes and must always be available (a customer should be able to add items even during failures). A cart fits in a single value; Riak's Set/Map CRDTs merge concurrent updates from multiple devices. Eventual consistency is acceptable; TTL clears abandoned carts.
User profile	**Riak**	Profile data is accessed by user ID key, is read often and updated rarely, has varying fields (flexible values), and needs massive scale and availability. No complex relationships or joins are needed.
Social-network recommendation	**Neo4j**	Requires exploring relationships (friends of friends, shared interests, mutual groups, liked posts) over many hops. Graph traversal is fast and Cypher expresses recommendation patterns simply; doing this in Riak would need many lookups and manual relationship management.

Conclusion: choose a key-value store such as Riak when data is accessed by a known key and requires scale and availability; choose a graph database such as Neo4j when the value of the data lies in its connections. Large real-world systems often use both (polyglot persistence): for example Riak for sessions, carts and profiles, and Neo4j for the recommendation engine, linked by user IDs.